

Лекция 3. Анализ сетевого трафика: tcpdump и Wireshark

Цель лекции

Понять пакетный анализ как способ увидеть фактический сетевой обмен

Выбирать правильную точку захвата, интерфейс и фильтр

Записывать PCAP-файл и открывать его в Wireshark

По ARP, DNS, TCP и HTTP определять, где ломается обмен

Оформлять краткий PCAP-отчет с наблюдением и выводом

Учебные вопросы

1. Анализ трафика, capture point, interface и promiscuous mode
2. Capture filters и Display filters
3. PCAP: запись, чтение и безопасное хранение
4. ARP и DNS
5. TCP handshake, RST, retransmission и timeout
6. HTTP и ограничения HTTPS

1. Пакетный анализ показывает реальный обмен

Пользователь видит проблему: сайт не открывается,
имя не разрешается, порт недоступен

tcpdump показывает, какие пакеты реально уходят и
приходят на интерфейс

Wireshark помогает разобрать сохраненный PCAP
удобнее и нагляднее

Packet capture дополняет ip route, ss и journalctl, но не
заменяет их

Главный вопрос анализа: на каком этапе обмен
перестал идти нормально?

tcpdump и Wireshark решают разные задачи

tcpdump - консольная утилита для захвата и чтения пакетов

Wireshark - графический анализатор пакетов и протоколов

tcpdump удобен на сервере без GUI и при удаленной диагностике по SSH

Wireshark удобен для просмотра последовательности пакетов и фильтрации

Типовой процесс: захватить tcpdump → сохранить PCAP → открыть в Wireshark

Capture Point определяет, какие факты мы увидим

Capture Point = место, где выполняется захват трафика

Если захват на client01, видно, что отправил и получил клиент

Если захват на web01, видно, дошел ли запрос до сервера

Если пакет виден в одной точке и не виден в другой, проблема находится между ними

Неправильная точка захвата может привести к неправильному выводу

Двухточечный захват помогает локализовать отказ

Request виден на клиенте, но не виден на сервере → проблема между ними

Request дошел до сервера, но ответа нет → service, firewall или application

Server отправил Reply, но client его не получил → reverse path или промежуточная сеть

Две точки захвата особенно полезны при споре: сеть виновата или сервер?

Вывод делают по совпадению времени, адресов, портов и направления пакетов

Интерфейс выбирают по маршруту к цели

`ip -br link` - показывает доступные сетевые интерфейсы

`ip route get 192.168.30.10` - показывает, какой интерфейс и source IP Linux выберет локально

После выбора интерфейса выполняют захват: `sudo tcpdump -i enp0s3 icmp`

Если выбран не тот интерфейс, нужный трафик может не попасть в захват

В отчете нужно указать `interface`, на котором выполнялся `capture`

Promiscuous Mode не превращает порт в зеркало сети

Promiscuous Mode - режим, в котором интерфейс передает ОС больше полученных кадров

Он помогает видеть кадры, которые пришли на данный интерфейс

Он не заставляет Ethernet-коммутатор отправлять чужой unicast-трафик на этот порт

Для чужого трафика могут понадобиться SPAN, Port Mirroring, TAP или capture на нужной VM

Детали VirtualBox и режимов гипервизора рассматриваются дополнительно

Схема: выбор точки захвата в серверной сети

Правильная точка захвата = правильные факты в PCAP

Легенда

●

 Хорошие точки захвата

○

 Дополнительные точки

⊗

 Точка почти бесполезна

-->

 Поток запроса

- ->

 Поток ответа



Как локализовать потерю с помощью двух точек захвата

🔍 Запрос виден на клиенте, но не виден на сервере	→	Потеря между клиентом и сервером	Проверяйте: сеть, firewall, маршрутизаторы, кабели, ACL, NAT
📄 Запрос пришел на сервер, но ответа нет	→	Проблема на сервере	Проверяйте: приложение, сервис, local firewall, лимиты, ресурсы
↩ Сервер отправил ответ, но клиент не получил	→	Проблема на обратном пути или у клиента	Проверяйте: обратный маршрут, асимметрия, потери, клиентский firewall
🛡 Ответ виден на firewall, но не виден на клиенте	→	Проблема в клиентской сети	Проверяйте: маршрутизаторы провайдера, Wi-Fi, кабели, firewall клиента

Рекомендации по выбору точки захвата

- ✓ Начинайте с логического выбора по маршрутизации: `ip route get <dest>`
- ✓ Захватывайте на обеих сторонах проблемного участка
- ✓ Выбирайте интерфейс, через который реально идет трафик
- ✓ Учитывайте режим сети в VirtualBox и настройки Promiscuous Mode
- ✓ Сохраняйте PCAP с пояснениями: где, когда и почему сделан захват

Полезные команды

```
ip -br link
ip route get 10.0.0.10
sudo tcpdump -i enp0s3 -nn
sudo tcpdump -i enp0s3 -w capture.pcap
wireshark capture.pcap
```



Правильная точка захвата = достоверные факты = быстрая диагностика



Неверная точка захвата может привести к ложным выводам!

2. Capture Filter ограничивает файл захвата

Capture Filter применяется во время захвата

Он использует BPF-синтаксис в tcpdump

Пакеты, которые не прошли capture filter, не попадут в PCAP

Вернуть отфильтрованный пакет после завершения захвата нельзя

Принцип: Capture Filter → ограничивает файл

Display Filter ограничивает только экран Wireshark

Display Filter применяется после захвата

Он скрывает лишние пакеты в интерфейсе Wireshark

Исходный PCAP при этом не изменяется

Скрытые пакеты можно снова показать, убрав или изменив фильтр

Принцип: Display Filter → ограничивает экран

Capture Filter и Display Filter нельзя путать

Capture Filter: используется во время capture

Display Filter: используется после capture

Capture Filter: tcpdump и BPF

Display Filter: Wireshark display syntax

Capture Filter не записывает лишние пакеты

Display Filter скрывает лишние пакеты, но оставляет их
в PCAP

Базовые Capture Filters покрывают большинство учебных задач

host 192.168.30.10 - трафик конкретного узла

net 192.168.30.0/24 - трафик подсети

port 53 - трафик DNS-порта

src host 192.168.10.5 - источник с указанным IP

dst host 192.168.30.10 - назначение с указанным IP

icmp, arp, udp, tcp - фильтр по протоколу

Фильтры src, dst и port отвечают на разные вопросы

src host - кто отправляет пакеты

dst host - кому отправляют пакеты

port 53 - трафик, где порт 53 может быть источником или назначением

tcp dst port 443 - TCP-пакеты, направленные на порт HTTPS

Выбор фильтра должен соответствовать вопросу диагностики

Слишком узкий capture filter может скрыть важные пакеты

Сложные BPF-фильтры лучше брать в кавычки

Пример: `sudo tcpdump -i enp0s3 'host 192.168.30.10
and (tcp port 80 or tcp port 443)'`

Кавычки защищают скобки и логические операторы от обработки оболочкой

`and` уточняет условие: должны выполняться обе части
`or` расширяет условие: подходит один из вариантов

В лабораторной лучше начинать с простого фильтра и постепенно уточнять его

Display Filters Wireshark помогают быстро найти нужные пакеты

`dns` - показать DNS-пакеты

`tcp.port == 80` - показать TCP-трафик порта 80

`ip.addr == 192.168.10.20` - показать пакеты с участием указанного IP

`http` - показать HTTP-пакеты

`dns.flags.response == 0` - показать DNS-запросы

`dns.flags.response == 1` и `dns.flags.rcode != 0` - показать ответы и DNS-ошибки

3. PCAP сохраняет сетевое событие для анализа

PCAP - файл с сохраненными пакетами сетевого захвата

Основная команда: `sudo tcpdump -i enp0s3 -nn -w test.pcap`

`-w test.pcap` записывает пакеты в файл

`-nn` отключает преобразование IP-адресов и портов в имена

PCAP позволяет анализировать проблему после завершения события

Файл можно открыть в Wireshark на рабочей станции преподавателя или студента

PCAP можно прочитать без Wireshark

`tcpdump -r test.pcap` - читает сохраненный файл

`tcpdump -nn -r test.pcap` - читает файл без разрешения имен и портов

Чтение через `tcpdump` полезно на сервере без графического интерфейса

Wireshark удобнее для поиска цепочек ARP, DNS, TCP и HTTP

Один PCAP можно анализировать несколько раз с разными `display filters`

Захват нужно ограничивать, чтобы файл не рос бесконечно

`sudo tcpdump -i enp0s3 -c 1000 -w test.pcap -`

остановится после 1000 пакетов

`sudo timeout 60 tcpdump -i enp0s3 -nn -w test.pcap -`

остановится через 60 секунд

Ограничение особенно важно на активном сервере

Дополнительно: `-C` и `-W` помогают ограничивать

размер и количество файлов

Дополнительно: `snaplen -s` управляет тем, сколько байт

каждого пакета сохраняется

Полезные ключи `tcpdump` нужны для чтения захвата

- nn - не разрешать IP-адреса и порты в имена
- v - показать больше подробностей
- e - показать Ethernet header, включая MAC-адреса
- A - показать ASCII payload, если протокол не зашифрован

Дополнительно: -X, -vvv и -tttt полезны при более глубоком анализе

Для junior важнее выбрать точку захвата и фильтр, чем запомнить все ключи

PCAP может содержать чувствительные данные

В PCAP могут попасть IP-адреса, DNS-имена, URL и Cookie

В незашифрованных протоколах могут попасть учетные данные

`sudo install -d -m 700 /root/pcap` - создает закрытый каталог для захватов

`sudo chmod 600 /root/pcap/test.pcap` - закрывает файл от лишнего чтения

PCAP нельзя отправлять в общий чат или наружу без проверки содержимого

В учебной группе PCAP рассматривают как

Путь от захвата tcpdump к анализу в Wireshark

1 Проблема

Нужно понять,
что происходит в сети



2 Выбрать точку захвата

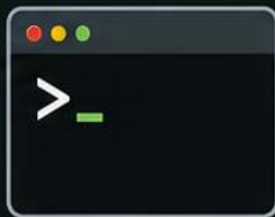
Клиент • Сервер •
Маршрутизатор



Клиент Сервер Маршрутизатор

3 tcpdump

Интерфейс +
capture filter



```
tcpdump -i enp0s3 -w test.pcap
```

4 PCAP-файл

Сохранённый
трафик



5 Wireshark

Открыть и применить
display filter



- arp
- dns
- tcp.port == 80

6 Вывод

Где ломается
ARP / DNS / TCP / HTTP



Результат анализа



ARP

Проверка
разрешения
адресов



DNS

Проверка
разрешения
имён



TCP

Проверка
соединения
и передачи



HTTP

Проверка
веб-трафика
и ответов



Главная идея



tcpdump — захват трафика



PCAP — сохранение



Wireshark — разбор



→ 👁 → ✓ Фильтр → наблюдение → вывод



Сначала **захватывают** трафик, затем **сохраняют PCAP**
и только после этого **анализируют** пакеты в Wireshark.

4. ARP показывает локальное соответствие IP и MAC

ARP (Address Resolution Protocol) - протокол поиска MAC-адреса по IPv4-адресу в локальной сети

ARP Request: кто имеет этот IP-адрес?

ARP Reply: этот IP-адрес имеет такой MAC-адрес

`sudo tcpdump -i enp0s3 -e arp` - показывает ARP с Ethernet-заголовком

`ip neigh` - показывает таблицу соседей Linux

ARP работает только в пределах локального L2-сегмента

ARP Request без Reply указывает на локальную проблему

Сценарий: ARP Request → нет ARP Reply

Проверить, включен ли соседний узел

Проверить, что интерфейс UP

Проверить правильную VLAN

Проверить IP-адрес и маску

Не нужно углубляться в ARP security, пока не проверены базовые причины

DNS показывает, удалось ли получить IP по имени

DNS (Domain Name System) - система преобразования доменных имен в IP-адреса

`sudo tcpdump -i enp0s3 'port 53'` - захватывает DNS-трафик на порту 53

UDP 53 - основной режим DNS-запросов

TCP 53 тоже используется DNS, например для крупных ответов или передачи зон

В Wireshark display filter `dns` быстро показывает DNS-пакеты

Если по IP работает, а по имени нет, DNS становится первой веткой проверки

DNS-ответы нужно читать по смыслу

NOERROR - запрос выполнен успешно

NXDOMAIN - такого имени не существует

SERVFAIL - DNS-сервер не смог обработать запрос

REFUSED - DNS-сервер отказал в обработке запроса

`dns.flags.response == 0` показывает запросы

`dns.flags.rcode != 0` помогает быстро найти DNS-ошибки

Дополнительно: не весь современный DNS виден на port 53

DoH и DoT могут шифровать DNS-трафик

В таком случае обычный capture filter port 53 может ничего не показать

Для базовой лаборатории достаточно классического DNS на UDP/TCP 53

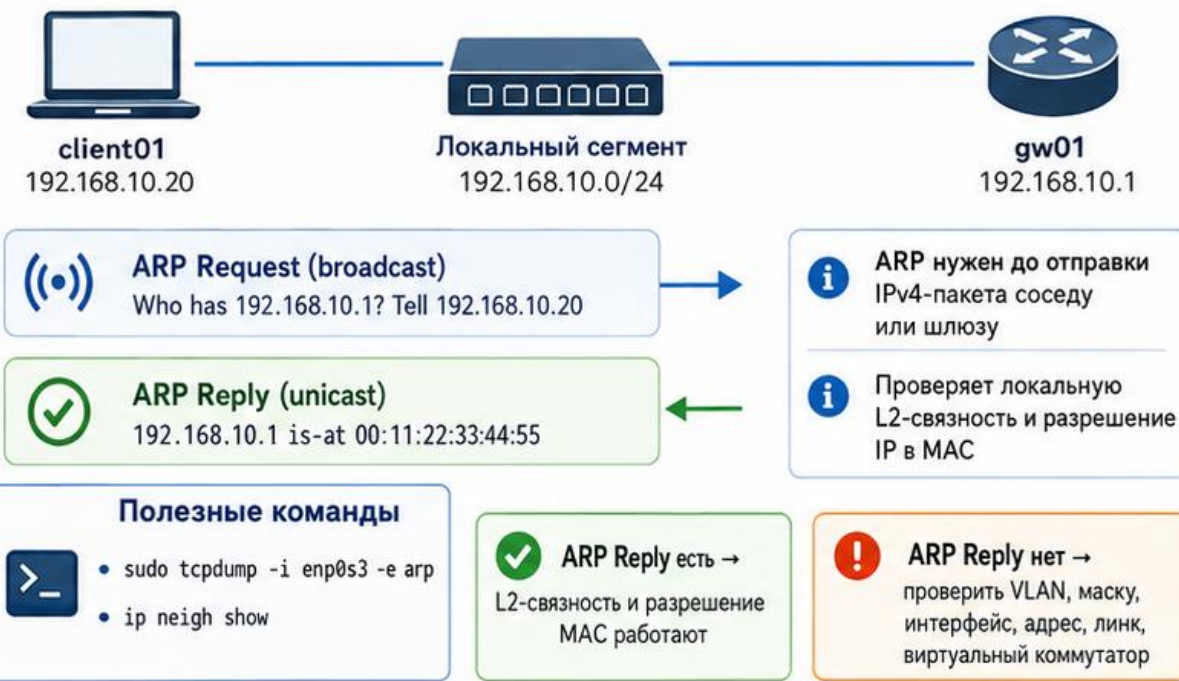
Главная задача junior - понять, есть ли запрос, есть ли ответ и какой код ответа

Глубокий анализ зашифрованного DNS относится к продвинутым темам

Схема: ARP и DNS в диагностике подключения

Сначала проверяем IP→MAC в локальном сегменте, затем имя→IP

ARP: есть ли MAC-адрес соседа или шлюза?



DNS: разрешается ли имя в IP?



Как читать симптомы

?	По IP доступ есть, по имени нет	→	Вероятна DNS-проблема
!	Нет ARP Reply к шлюзу	→	L2/VLAN/адресная проблема
!	DNS Query уходит, но Response нет	→	DNS-сервер недоступен или трафик блокируется
✗	DNS отвечает NXDOMAIN	→	Неверное имя или отсутствует DNS-запись

5. TCP Handshake показывает установку соединения

TCP (Transmission Control Protocol) - протокол надежной передачи данных с установлением соединения

Успешный handshake: Client SYN → Server SYN/ACK → Client ACK

Если handshake успешен, TCP-порт доступен на сетевом уровне

Успешный TCP handshake еще не доказывает корректную работу приложения

Для захвата HTTP-подключения: `sudo tcpdump -i enp0s3 -nn 'host 192.168.10.20 and tcp port 80'`

Open, Reject и Drop выглядят по-разному

OPEN: SYN → SYN/ACK → ACK

CLOSED или REJECT: SYN → RST

DROP: SYN → SYN → SYN, ответа нет

RST означает активный резкий сброс соединения

Timeout означает, что ответ не пришел вовремя

Timeout не является TCP-флагом

FIN, RST и Timeout отвечают на разные вопросы

FIN - нормальное завершение TCP-соединения

RST - активный сброс соединения одной из сторон

Timeout - ожидание истекло, потому что нужного ответа не было

RST часто указывает на закрытый порт или отказ приложения

Timeout часто заставляет проверить route, firewall, ACL или потерю пакетов

Диагноз делают по последовательности пакетов, а не по одному слову

Retransmission и Duplicate ACK изучают обзорно

Retransmission - повторная отправка TCP-сегмента

Duplicate ACK - повторное подтверждение уже полученного сегмента

Много таких событий может говорить о loss, delay или congestion

Для junior достаточно распознать признак нестабильного обмена

Глубокий анализ Sequence Number в этой лекции не требуется

Дополнительно: offloading может менять вид локального PCAP

TSO, GSO, GRO и checksum offload относятся к сетевому offloading

Из-за offloading локальный PCAP иногда выглядит необычно

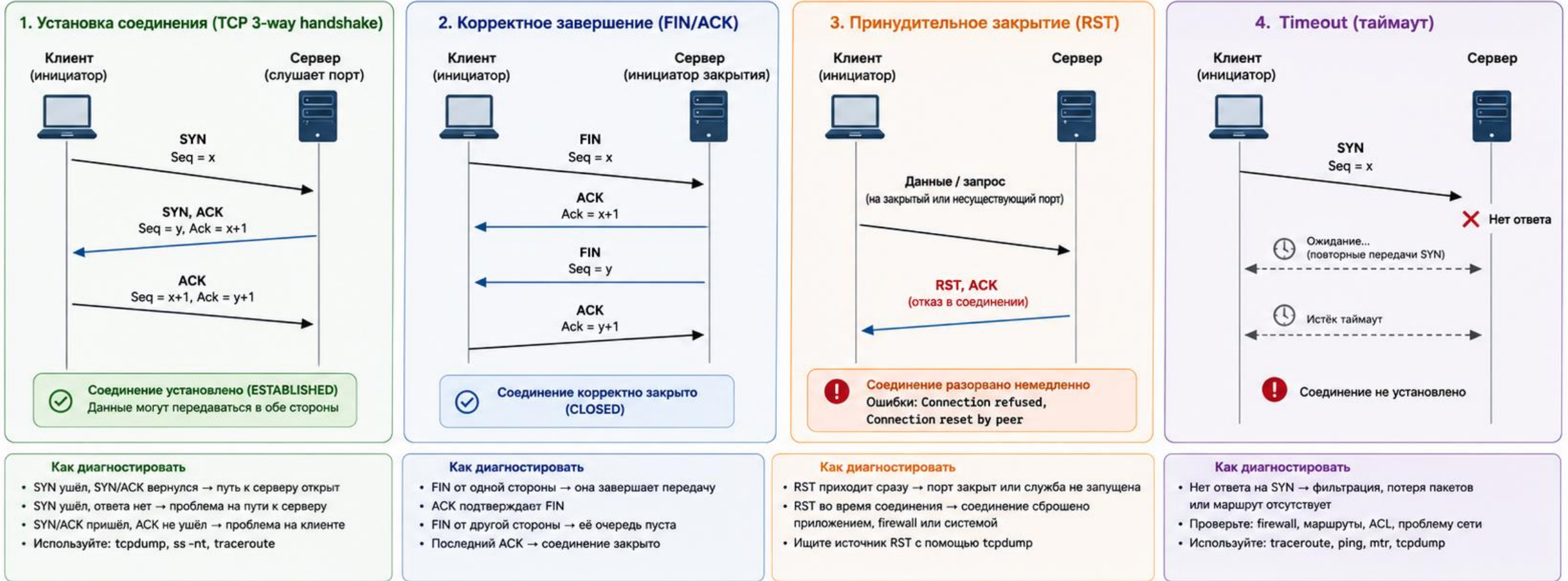
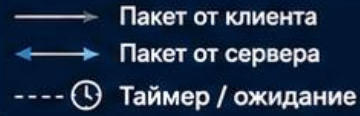
Например, анализатор может показывать странные checksum-сообщения

В базовой лаборатории offloading не является основной темой

Сначала нужно доказать ARP, DNS, TCP и HTTP, а уже потом разбирать особенности драйвера

Диаграмма: TCP SYN, FIN, RST и timeout

Флаги TCP помогают понять состояние соединения и причину проблемы



Ключевые TCP флаги

- SYN** – синхронизация номеров последовательности, запрос на соединение
- ACK** – подтверждение получения (Ack = номер следующего ожидаемого байта)
- FIN** – завершение передачи данных (больше нечего отправлять)
- RST** – сброс соединения (ошибка, недопустимое состояние или политика)
- PSH/URG** – служебные флаги (данные немедленно / срочные данные)

Полезные команды

```
# Захват трафика
sudo tcpdump -ni any tcp port 80

# Просмотр соединений
ss -nt state all

# Проверка маршрута
ip route get <IP-адрес>
```

Что смотреть в Wireshark

- Флаги в колонке Info: [S],[S.],[.],[F.],[R],[R.]
- Номера последовательности (Seq) и подтверждения (Ack)
- Window size, retransmission, duplicate ACK
- RTT (Round Trip Time) и время ожидания

Подсказка

Пакетный анализ показывает фактическое поведение сети, а не то, что «должно быть». Два захвата (клиент + сервер) дают полную картину!

6. HTTP можно читать как прикладной протокол поверх TCP

HTTP (HyperText Transfer Protocol) - прикладной протокол передачи веб-запросов и ответов

Пример запроса: GET /index.html HTTP/1.1

Заголовок Host: web01.company.test показывает имя сайта, к которому обращается клиент

Пример ответа: HTTP/1.1 200 OK

Wireshark display filter http показывает HTTP-пакеты

Если HTTP-ответ получен, TCP и HTTP дошли до приложения

HTTP-код помогает разделить сеть от приложения

200 - запрос успешно обработан

301/302 - redirect на другой URL

404 - сервер работает, но ресурс не найден

500 - сервер ответил, но ошибка возникла в
приложении

HTTP code уже доказывает, что TCP-соединение и
HTTP-обмен состоялись

После HTTP 404 или 500 диагностика уходит в веб-
сервер и приложение, а не в ARP

HTTPS скрывает содержимое HTTP внутри TLS

HTTPS - это HTTP внутри TLS-шифрования

Обычно видны IP-адреса, TCP 443, время пакетов, размеры пакетов и TCP handshake

Часть TLS handshake тоже может быть видна, но содержимое HTTP скрыто

Для junior важно определить, есть ли соединение и где оно ломается

Расшифровка TLS не является целью этой лекции

Дополнительно: расшифровка HTTPS требует контролируемой среды

В лаборатории HTTPS иногда можно расшифровать при наличии контролируемого клиента и ключевого материала

Темы PFS, ephemeral DH, SSLKEYLOGFILE и TLS proxy относятся к продвинутому анализу

Нельзя пытаться расшифровывать чужой трафик без разрешения

Для базовой диагностики достаточно видеть TCP 443 и признаки успешного или неуспешного соединения

Если нужен анализ приложения, лучше использовать серверные журналы и контролируемую тестовую

Единый сценарий: client01 обращается к web01

client01: 192.168.10.10

web01: 192.168.10.20

dns01: 192.168.30.53

ARP: `sudo tcpdump -i enp0s3 -e arp`

DNS: `sudo tcpdump -i enp0s3 'port 53'`

TCP/HTTP: `sudo tcpdump -i enp0s3 -nn 'host 192.168.10.20 and tcp port 80'`

PCAP сценария сохраняют для Wireshark

Команда: `sudo tcpdump -i enp0s3 -nn -w web-test.pcap 'host 192.168.10.20'`

После воспроизведения проблемы захват останавливают

Файл web-test.pcap открывают в Wireshark

Сначала ищут ARP и DNS, затем TCP handshake, затем HTTP

Итоговый вывод должен объяснять, на каком этапе обмен сломался

Break/Fix: четыре понятных сценария анализа

ARP Problem: Request → no Reply

DNS Problem: Query → NXDOMAIN или Query → no Response

TCP Problem: SYN → RST или SYN → retransmission → timeout

HTTP Problem: GET → HTTP 404 или GET → HTTP 500

Для каждого сценария студент указывает capture point, filter, наблюдение и вывод

Исправление проверяется повторным захватом или повторным запросом

PCAP-отчет должен ссылаться на конкретные пакеты

Симптом - что не работает у пользователя

Capture Point - где выполнялся захват

Interface - на каком интерфейсе был tcpdump

Capture Filter - каким фильтром ограничен захват

Время, наблюдение, вывод и дополнительная
проверка

Результат после исправления - что подтвердило
восстановление обмена

Пример строки PCAP-отчета

Симптом: client01 не открывает

http://web01.company.test

Capture Point: client01, interface enp0s3

Filter: host 192.168.10.20 and tcp port 80

Наблюдение: SYN повторяется, SYN/ACK не приходит

Вывод: TCP-соединение до web01:80 не

устанавливается, вероятен firewall, route или

недоступный сервис

Дополнительная проверка: capture на web01 и ss -ln

| grep :80

Итоги лекции

Пакетный анализ показывает фактический обмен, но не заменяет системную диагностику

Capture Point и interface определяют, какие пакеты попадут в наблюдение

Capture Filter ограничивает файл, Display Filter ограничивает экран

PCAP нужно ограничивать по объему и хранить как чувствительный диагностический файл

ARP, DNS, TCP и HTTP позволяют найти этап, где ломается обмен

HTTPS на базовом уровне анализируют по

Контрольные вопросы

1. Для чего используется tcpdump?
2. Чем tcpdump отличается от Wireshark?
3. Почему важна точка захвата?
4. Чем Capture Filter отличается от Display Filter?
5. Что означает ARP Request без Reply?
6. Как выглядит успешный TCP Handshake?
7. Чем RST отличается от Timeout?
8. Что означает повторный SYN без ответа?
9. Что можно увидеть в обычном HTTP?
10. Почему содержимое HTTPS обычно не видно?